

Rust Control Flow, Sorting, and Tooling

Grounded in ECM2433 rust_01_tooling, rust_02_crash_course, rust_08_closures_iterators, and Rust worksheets on sorting practical data.

What You Need To Know

1. Tooling: *rustc*, *Cargo*, *rust-analyzer*, *fmt*, and *clippy*

rustc is the compiler. *cargo* is the build system and package manager. *Cargo* replaces manual Makefiles/CMake for normal Rust projects and manages building, running, testing, and dependencies.

```
cargo new demo
cargo check
cargo build
cargo run
cargo test
cargo fmt
cargo clippy
```

- `cargo check` type-checks quickly without producing the final binary.
- `cargo fmt` formats code in standard Rust style.
- `cargo clippy` runs extra lints and catches common mistakes.
- `rust-analyzer` is the editor language server: autocomplete, inline errors, go-to-definition, and type hints.

2. Functions, returns, and *unit*

Rust functions use `fn`. Parameters need type annotations. Return types go after `->`. If no return type is written, the function returns *unit*, written `()`.

```
fn double(n: i32) -> i32 {
    n * 2           // no semicolon: final expression is returned
}

fn print_value(n: i32) {
    println!("{n}"); // returns ()
}
```

A semicolon turns an expression into a statement. That matters because a block returns its final expression.

3. *if* expressions and loop control

`if` is an expression, so it can produce a value. The condition must be `bool`; Rust has no C-style truthy integers.

```
let label = if score >= 40 {
    "pass"
} else {
    "fail"
};
```

Rust has `loop`, `while`, and `for`. `continue` skips to the next iteration. `break` exits a loop. A `loop` can return a value with `break value`.

```
let mut n = 0;
let answer = loop {
    n += 1;
    if n == 3 {
        break n * 10;
    }
};
```

4. Loop labels and doc comments

Loop labels let `break` or `continue` target an outer loop. Doc comments use `///` before the item they document.

```
'outer: for row in 0..3 {
    for col in 0..3 {
        if row + col > 3 {
            break 'outer;
        }
    }
}

/// Returns true if the mark is a pass.
fn is_pass(mark: u32) -> bool {
    mark >= 40
}
```

5. Sorting practical data

For basic values, `sort` is enough. For custom ordering, use `sort_by` with a closure. Integer and string values implement `Ord`, so use `cmp`. Floating-point values use `partial_cmp` because NaN means not every pair has a total

ordering.

```
let mut teams = vec![
    (5, "pear"),
    (5, "apple"),
    (2, "orange"),
];

teams.sort_by(|(count_a, word_a), (count_b, word_b)| {
    count_b
    .cmp(count_a)
    .then_with(|| word_a.cmp(word_b))
});
```

This sorts by count descending, then word ascending. `then_with` only runs the tie-breaker if the first comparison is equal.

6. Sorting floats with `partial_cmp`

```
let mut measurements = vec![
    (0.25, 2),
    (0.25, 0),
    (0.5, 1),
];

measurements.sort_by(|(value_a, id_a), (value_b, id_b)| {
    value_a
    .partial_cmp(value_b)
    .unwrap()
    .then_with(|| id_a.cmp(id_b))
});
```

`partial_cmp` returns `Option<Ordering>`. In controlled worksheet data with ordinary finite floats, `unwrap()` is usually acceptable. In production code, consider how to handle NaN.

Practice Questions

Question 1 - choose the Cargo command:

```
// You want fast compiler feedback but do not need a runnable binary yet.
```

- 1 Give the Cargo command and explain why it fits.

Question 2 - identify rust-analyzer's role:

```
// VS Code shows inline Rust errors and type hints while you type.
```

- 2 Name the tool normally responsible for this editor support and say what it is not responsible for.

Question 3 - fix the return value:

```
fn double(n: i32) -> i32 {
    n * 2;
}
```

- 3 Explain why this does not return an `i32`, then fix it.

Question 4 - use `if` as an expression:

```
let score = 72;
let result = /* pass if score >= 40, otherwise fail */;
```

- 4 Complete the expression so `result` is either `"pass"` or `"fail"`.

Question 5 - break with a value:

```
let mut n = 0;
let found = loop {
    n += 1;
    if n * n > 20 {
        /* return n from the loop */
    }
};
```

- 5 Complete the missing line.

Question 6 - labelled break:

```
'outer: for row in 0..4 {
    for col in 0..4 {
        if row + col == 5 {
            /* exit both loops */
        }
    }
}
```

- 6 Complete the statement that exits both loops.

Question 7 - doc comment placement:

```
// Add a doc comment for this function.
fn is_even(n: i32) -> bool {
    n % 2 == 0
}
```

7 Write the doc comment line in the correct place.

Question 8 - sort counts with a tie-breaker:

```
let mut items = vec![
    (2, "red"),
    (5, "blue"),
    (5, "amber"),
];

/* sort by count descending, then word ascending */
```

8 Write the `sort_by` call.

Question 9 - sort floating-point values:

```
let mut readings = vec![(0.5, 2), (0.1, 9), (0.5, 1)];

/* sort by float ascending, then id ascending */
```

9 Write the `sort_by` call using `partial_cmp`.

Question 10 - top n after sorting:

```
let mut scores = vec![
    (String::from("Ada"), 12),
    (String::from("Grace"), 18),
    (String::from("Linus"), 15),
];

/* return the top 2 scores, highest first */
```

10 Sort the vector and produce a `Vec` containing the top two entries.

Mark Scheme

Attempt all questions before checking.

- 1 `cargo` check. It type-checks quickly without producing the final binary.
- 2 `rust-analyzer`. It provides editor/language-server support; it is not Cargo and it does not replace the compiler.
- 3 The semicolon makes `n * 2` a statement, so the block returns `()`. Fix by removing the semicolon or writing `return n * 2;`.
- 4 `let result = if score >= 40 { "pass" } else { "fail" };`
- 5 `break n;`
- 6 `break 'outer;`
- 7 Place `/// Returns true when n is even.` immediately before `fn is_even`.
- 8 Expected pattern: `items.sort_by(|(count_a, word_a), (count_b, word_b)| count_b.cmp(count_a).then_with(|| word_a.cmp(word_b)))`;
- 9 Expected pattern: `readings.sort_by(|(value_a, id_a), (value_b, id_b)| value_a.partial_cmp(value_b).unwrap().then_with(|| id_a.cmp(id_b)))`;
- 10 Expected pattern: sort by descending score, then use `into_iter().take(2).collect::<Vec<_>>()`.

Useful Pattern for Question 10

```
scores.sort_by(|(_, score_a), (_, score_b)| score_b.cmp(score_a));

let top_two: Vec<_> = scores
    .into_iter()
    .take(2)
    .collect();
```